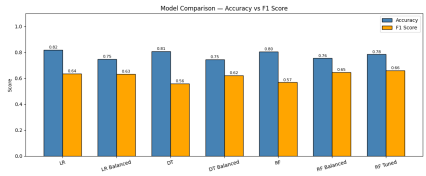
Customer Churn Prediction

Using Machine Learning



Submitted in partial fulfillment of the requirements for the course: Artificial Intelligence

Department:	Data Science
Semester:	6th Semester
University:	Your University Name
Dataset:	Telco Customer Churn (IBM)
Tools Used:	Python, Scikit-learn, Streamlit

Group Members

Sr.	Name	Roll No
1	Abdullah Qasim	23L-2640
2	Muhammad Maaz	23L-2579

Contents

Abstract

Customer churn prediction is a critical problem in the telecommunications industry, where retaining existing customers is significantly more cost-effective than acquiring new ones. This project develops a machine learning pipeline to predict whether a customer will churn (leave the service) based on demographic, account, and service usage features. The IBM Telco Customer Churn dataset containing 7,043 customer records and 21 features was used. Multiple classification algorithms were trained and evaluated, including Logistic Regression, Decision Tree, and Random Forest, along with class-balanced and hyperparameter-tuned variants. The best performing model — Random Forest (Balanced) — achieved the highest churn recall of 0.85 and ROC-AUC of 0.866, making it the most suitable model for this business-critical imbalanced classification problem. An interactive web application was developed using Streamlit to allow real-time churn prediction for new customer inputs.

Keywords: Customer Churn, Random Forest, Logistic Regression, Decision Tree, Class Imbalance, Scikit-learn, Streamlit.

1 Introduction

Customer churn refers to the phenomenon where customers stop doing business with a company. In the telecommunications sector, churn is a major concern as the cost of acquiring a new customer is typically five to ten times higher than retaining an existing one. Identifying customers who are likely to churn enables companies to proactively intervene through targeted retention strategies, such as personalized offers or improved service plans.

Machine learning provides powerful tools to address this problem by learning patterns from historical customer data and predicting future churn behavior. This project implements a complete end-to-end machine learning pipeline that includes:

- Exploratory Data Analysis (EDA) to understand data distributions and relationships
- Data preprocessing including missing value imputation, encoding, and scaling
- Training and evaluation of multiple machine learning models
- Handling of class imbalance using class weighting strategies
- Hyperparameter optimization using `RandomizedSearchCV`
- Model evaluation using accuracy, F1 score, ROC-AUC, recall, and learning curves
- Deployment of the best model through a Streamlit web application

1.1 Objectives

1. To preprocess and analyze the Telco Customer Churn dataset
2. To train and compare multiple classification models
3. To address class imbalance in churn prediction
4. To identify the most important features driving customer churn
5. To deploy a usable prediction interface for end users

2 Dataset Description

2.1 Overview

The dataset used in this project is the **IBM Telco Customer Churn** dataset, a widely used benchmark dataset for binary classification in customer analytics. It contains information about a telecommunications company's customers and whether they left within the last month.

Table 1: Dataset Summary

Property	Value
Total Records	7,043 customers
Total Features	21 columns
Target Variable	Churn (Yes / No)
Missing Values	11 (in TotalCharges column only)
Duplicate Rows	0

2.2 Feature Description

The dataset contains three categories of features:

Customer Demographics:

- `gender` — Male or Female
- `SeniorCitizen` — Whether the customer is a senior citizen (1 or 0)
- `Partner` — Whether the customer has a partner
- `Dependents` — Whether the customer has dependents

Account Information:

- `tenure` — Number of months the customer has been with the company (0–72)
- `Contract` — Contract type: Month-to-month, One year, Two year
- `PaperlessBilling` — Whether the customer uses paperless billing
- `PaymentMethod` — Electronic check, Mailed check, Bank transfer, Credit card
- `MonthlyCharges` — Monthly charge amount ($18.25 – $118.75)
- `TotalCharges` — Total amount charged to the customer

Services Subscribed:

- `PhoneService`, `MultipleLines`, `InternetService`
- `OnlineSecurity`, `OnlineBackup`, `DeviceProtection`
- `TechSupport`, `StreamingTV`, `StreamingMovies`

2.3 Statistical Summary

Table 2: Descriptive Statistics of Numeric Features

Feature	Count	Mean	Std	Min	Median	Max
SeniorCitizen	7043	0.162	0.369	0	0	1
tenure	7043	32.37	24.56	0	29	72
MonthlyCharges	7043	64.76	30.09	18.25	70.35	118.75

3 Exploratory Data Analysis

3.1 Class Distribution

The dataset exhibits class imbalance, with 73.46% of customers labeled as *No Churn* and 26.54% labeled as *Churn*. This imbalance must be addressed during model training to prevent the model from being biased toward the majority class.

Table 3: Churn Class Distribution

Class	Count	Percentage
No Churn (0)	5,174	73.46%
Churn (1)	1,869	26.54%
Total	**7,043**	**100%**

3.2 Feature Distributions

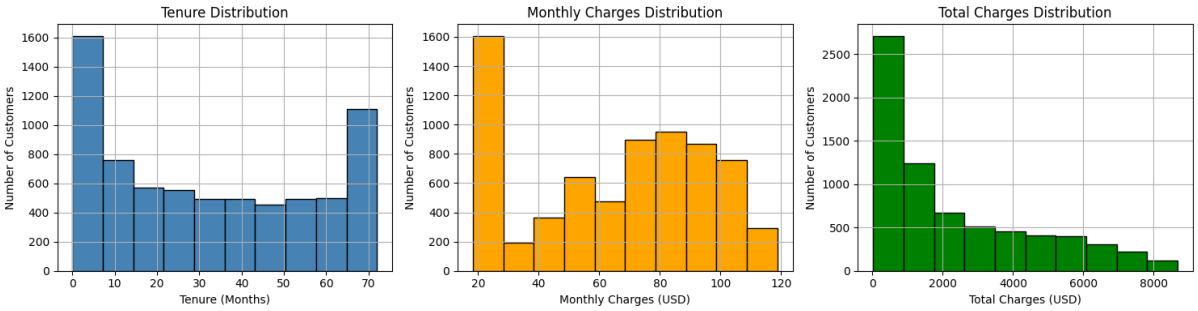


Figure 1: Distribution of Numeric Features: Tenure, Monthly Charges, and Total Charges

Tenure Distribution: Shows a U-shaped pattern. A large spike at 0–10 months indicates many new customers, while another spike at 70+ months represents long-term loyal customers. Customers in the middle range (20–60 months) are fewer, suggesting this group has higher churn risk.

Monthly Charges Distribution: A bimodal distribution is observed. A large group of customers pay low charges ($20–$30), while another significant group pays high charges ($60–$100). This suggests two distinct customer segments: basic and premium plan users.

Total Charges Distribution: Heavily right-skewed. Most customers have low total charges because they are relatively new. A small group of long-term customers have accumulated charges up to $8,500+.

3.3 Churn by Contract Type

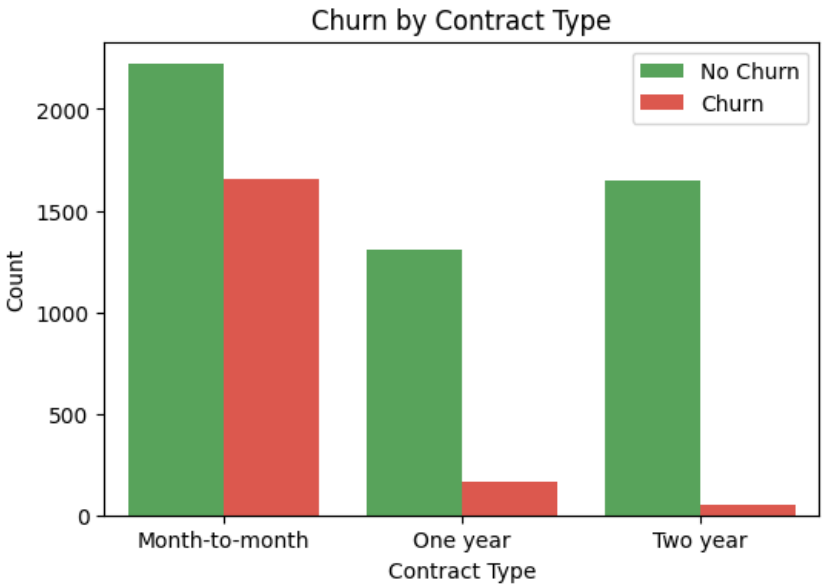


Figure 2: Churn Count by Contract Type

Contract type is one of the strongest predictors of churn. Month-to-month contract customers show a very high churn count compared to one-year and two-year contract customers, who demonstrate much higher loyalty. Customers on longer contracts have made a greater commitment to the service.

3.4 Monthly Charges vs Churn

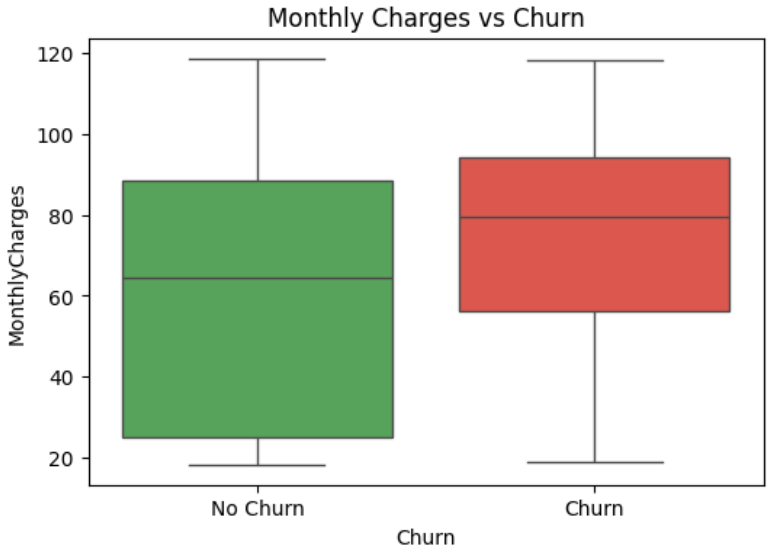


Figure 3: Distribution of Monthly Charges for Churned vs Non-Churned Customers

The boxplot reveals that churned customers have a notably higher median monthly charge (˜$80) compared to non-churned customers (˜$65). This indicates that customers paying

higher monthly fees are more likely to leave, possibly due to dissatisfaction with the price-to-value ratio.

3.5 Correlation Heatmap

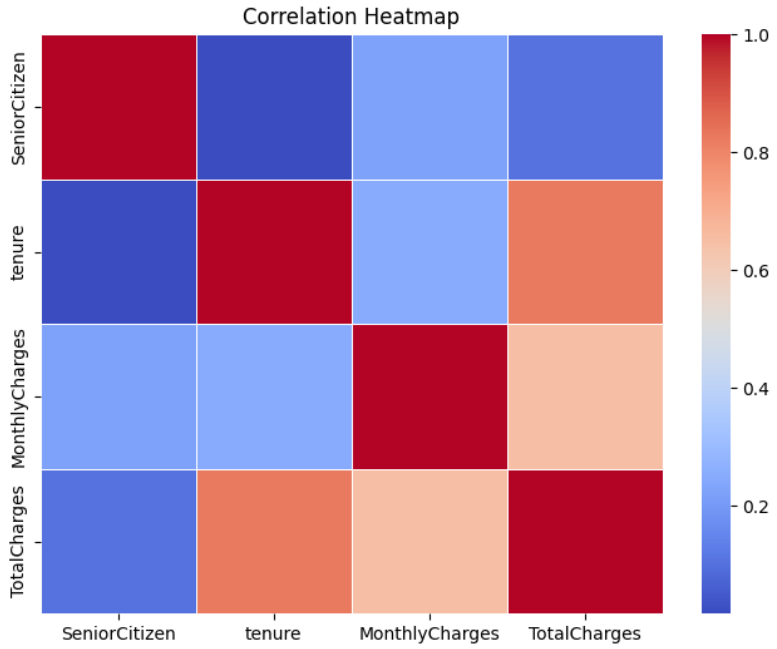


Figure 4: Correlation Heatmap of Numeric Features

Key observations from the correlation heatmap:

- `tenure` and `TotalCharges` have a strong positive correlation (0.83), which is expected since longer-tenured customers accumulate more total charges
- `MonthlyCharges` and `TotalCharges` are moderately correlated
- `SeniorCitizen` has a weak negative correlation with `tenure`

4 Data Preprocessing

4.1 Data Type Correction

The `TotalCharges` column was stored as an `object` type due to whitespace entries instead of numeric values. It was converted to `float64` using `pd.to_numeric()` with `errors='coerce'`, which converted invalid entries to `NaN` for subsequent handling.

4.2 Missing Value Treatment

After type conversion, 11 missing values were found in the `TotalCharges` column. These were filled using the column's **median** value, which is robust to outliers and appropriate for right-skewed data.

Table 4: Missing Value Summary

Column	Missing Before	Missing After
TotalCharges	11	0
All other columns	0	0

4.3 Duplicate Records

A check for duplicate rows confirmed that the dataset contained **zero duplicates**, so no records needed to be removed.

4.4 Feature Removal

The `customerID` column was removed as it is a unique identifier that carries no predictive information.

4.5 Feature Encoding

Target Variable: The `Churn` column was encoded as: `Yes` $\rightarrow$ 1, `No` $\rightarrow$ 0.

Binary Columns: The following columns were label-encoded:

`gender, Partner, Dependents, PhoneService, PaperlessBilling`

Multi-class Columns: One-Hot Encoding with `drop_first=True` was applied to prevent multicollinearity. After encoding, the dataset expanded from 21 to 31 columns.

4.6 Outlier Analysis

A boxplot analysis was performed on the three continuous numeric features: `tenure`, `MonthlyCharges`, and `TotalCharges`. No extreme outliers were detected. High values in `TotalCharges` correspond to genuine long-tenure customers with extended service histories, not data errors. Therefore, **no outliers were removed**.

4.7 Train-Test Split and Feature Scaling

The dataset was split into 80% training (5,634 samples) and 20% testing (1,409 samples) using `train_test_split` with `random_state=42` for reproducibility. `StandardScaler` was applied to normalize all features — fitted on the training set and applied to both training and test sets to prevent data leakage.

5 Model Development

5.1 Models Trained

Three classification algorithms were implemented, each in two variants — standard and class-balanced:

1. **Logistic Regression** — A linear model used as a baseline
2. **Decision Tree Classifier** — A non-linear tree-based model
3. **Random Forest Classifier** — An ensemble of decision trees

Additionally, a **RandomizedSearchCV** tuned Random Forest was trained as the final optimized model.

5.2 Handling Class Imbalance

Since the dataset is imbalanced (73% vs 27%), the `class_weight='balanced'` parameter was used in balanced model variants. This parameter adjusts the loss function to give higher weight to the minority class (churners), helping the model detect them more effectively.

5.3 Hyperparameter Tuning

The following constraints were applied to prevent overfitting in Decision Tree and Random Forest models:

Table 5: Hyperparameters Applied to Tree-Based Models

Parameter	Value
`max_depth`	5 (DT), 6 (RF)
`min_samples_split`	10–13
`min_samples_leaf`	10
`n_estimators`	100 (RF)

RandomizedSearchCV was applied to find the globally optimal Random Forest configuration:

Table 6: Best Parameters Found by RandomizedSearchCV

Parameter	Best Value
`n_estimators`	200
`max_depth`	10
`min_samples_split`	2
`min_samples_leaf`	4
`class_weight`	balanced

The search used 5-fold cross-validation with F1 score as the optimization metric, running 20 random iterations.

6 Results and Evaluation

6.1 Model Comparison

Table 7: Performance Comparison of All Models

Model	Accuracy	F1 (Churn)	ROC-AUC	Recall (Churn)
Logistic Regression	0.820	0.636	0.862	0.60
LR (Balanced)	0.747	0.633	0.862	0.82
Decision Tree (Tuned)	0.808	0.559	0.850	0.46
DT (Balanced)	0.745	0.622	0.836	0.79
Random Forest (Tuned)	0.805	0.568	0.866	0.49
RF (Balanced)	**0.755**	**0.648**	**0.866**	**0.85**
RF (RandomizedSearchCV)	0.785	0.660	0.865	0.79

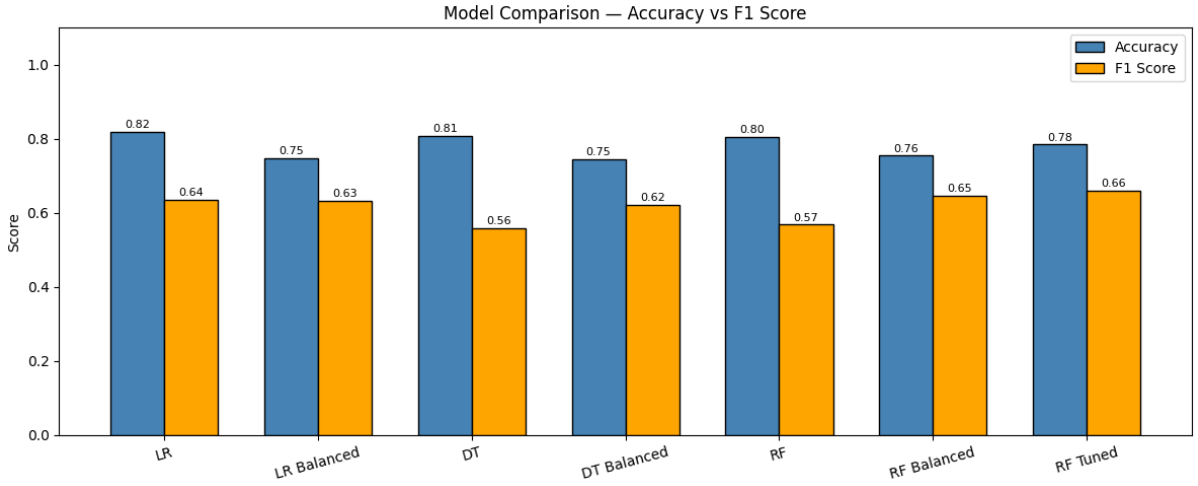


Figure 5: Model Comparison — Accuracy vs F1 Score for All Models

6.2 Best Model: RF (Balanced)

The Random Forest (Balanced) model was selected as the best model because it achieved the highest churn recall of **0.85**, correctly identifying 317 out of 373 churners. In an imbalanced churn prediction problem, recall is the priority metric since missing a real churner results in direct revenue loss for the business. Its classification report is:

Table 8: Classification Report — RF (Balanced)

Class	Precision	Recall	F1-Score	Support
No Churn (0)	0.93	0.72	0.81	1036
Churn (1)	0.52	0.85	0.65	373
Accuracy		0.755		1409
Macro Avg	0.73	0.79	0.73	1409
Weighted Avg	0.82	0.76	0.77	1409

Confusion Matrix Interpretation:

- The model correctly identified **317 out of 373 churners** (recall = 0.85) — the highest among all models
- Only **56 real churners** were missed — the lowest miss count of all models
- Precision for churn (0.52) means some loyal customers are incorrectly flagged, but this is acceptable since offering a retention discount to a loyal customer costs far less than losing a real churner

6.3 ROC-AUC Analysis

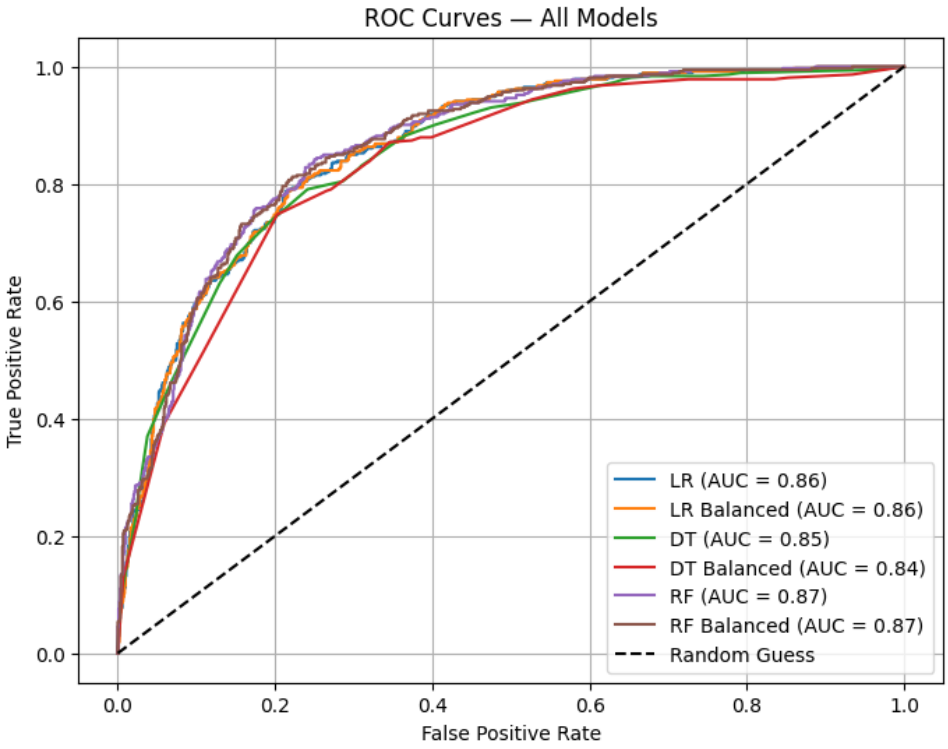


Figure 6: ROC Curves for All Models

All models performed significantly better than random guessing (AUC = 0.50). Random Forest models achieved the highest AUC of **0.866**, indicating strong discriminative ability between churners and non-churners. Logistic Regression also performed competitively with AUC = 0.862, demonstrating that even a simple linear model captures most of the predictive signal in this dataset.

6.4 5-Fold Cross-Validation Results

Table 9: Cross-Validation F1 Scores (5-Fold)

Model	Mean F1	Std Dev
Logistic Regression	0.592	0.018
LR (Balanced)	0.622	0.015
Decision Tree	0.568	0.029
DT (Balanced)	0.611	0.011
Random Forest	0.547	0.032
RF (Balanced)	**0.624**	**0.017**
RF (RandomizedSearchCV)	0.632	0.011

The RF (Balanced) model achieved a strong cross-validation F1 of **0.624** with a low standard deviation of **0.017**, confirming consistent and stable performance across all folds. While RF (RandomizedSearchCV) scored slightly higher F1 (0.632), the RF (Balanced) model was selected as best due to its superior recall on the churn class, which is the priority metric for this problem.

6.5 Feature Importance

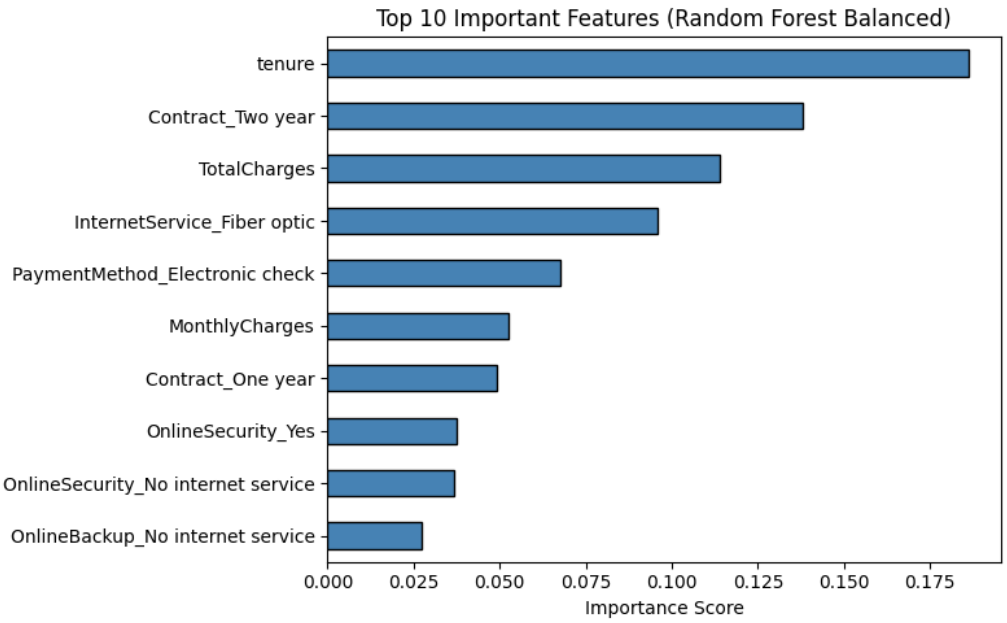


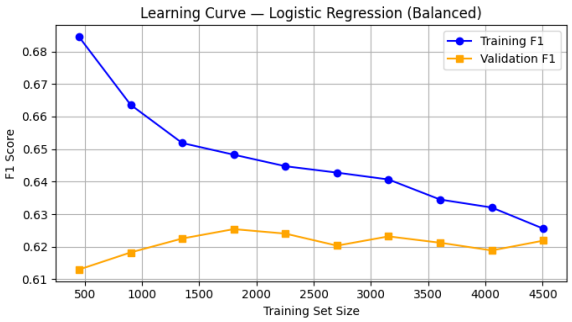
Figure 7: Top 10 Important Features (Random Forest Balanced)

The feature importance analysis revealed the following key drivers of churn:

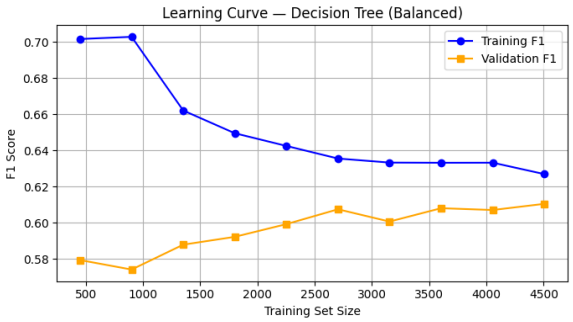
1. **tenure** — Most important feature. Longer tenure strongly predicts loyalty
2. **Contract_Two year** — Two-year contract customers rarely churn
3. **TotalCharges** — Closely related to tenure; higher charges indicate longer relationship

4. **InternetService_Fiber optic** — Fiber optic users show higher churn rates
5. **PaymentMethod_Electronic check** — Electronic check users churn more frequently
6. **MonthlyCharges** — Higher monthly charges correlate with churn
7. **Contract_One year** — One-year contracts provide moderate retention
8. **OnlineSecurity_Yes** — Having online security reduces churn likelihood

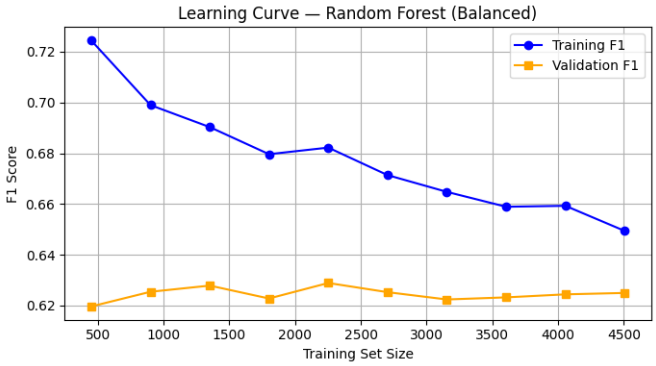
6.6 Learning Curve Analysis



(a) Logistic Regression (Balanced)



(b) Decision Tree (Balanced)



(c) Random Forest (Balanced)

Figure 8: Learning Curves for Three Models

Logistic Regression (Balanced): Training F1 decreases smoothly from 0.69 to 0.63 while validation F1 rises from 0.61 to 0.62. The very small gap (˜0.01) at the end indicates an excellent fit with no overfitting. The model has reached its learning capacity on this dataset.

Decision Tree (Balanced): Training F1 decreases from 0.70 to 0.63 and validation F1 rises from 0.58 to 0.61. The gap (˜0.02) is minimal and lines are converging, indicating a good fit. The slight zigzag pattern in validation suggests mild variance but remains acceptable.

Random Forest (Balanced): Training F1 decreases from 0.72 to 0.65 while validation F1 remains relatively stable around 0.62–0.63. A gap of ˜0.04 persists, indicating mild

overfitting. However, this is acceptable for a random forest on this dataset and the gap is well within tolerable limits.

Overall, all models show healthy learning behavior — a significant improvement over initial untuned models which exhibited severe overfitting (training F1 = 1.0, validation F1 = 0.50).

7 Application: Streamlit Web Interface

7.1 Overview

A web-based prediction application was developed using **Streamlit** to provide a user-friendly interface for real-time churn prediction. The application loads the trained `rf_bal` model (RF Balanced) and the fitted `StandardScaler` from saved `.pkl` files.

7.2 Application Features

- **Interactive Input Form:** Dropdown menus and sliders for all 30 customer features
- **Real-time Prediction:** Instant churn/no-churn prediction on button click
- **Risk Level Assessment:** Automatically categorizes the customer as High Risk, Medium Risk, or Low Risk based on predicted probability
- **Clean UI:** Two-column layout for easy data entry

7.3 Technical Workflow

1. User enters customer data through the interface
2. Input is encoded using the same binary and one-hot encoding logic applied during training
3. Encoded features are scaled using the saved `StandardScaler`
4. The scaled input is passed to `rf_bal.predict()` for prediction
5. Result is displayed as Churn / No Churn with a risk level indicator

7.4 Running the Application

```
# Install dependencies
pip install streamlit joblib scikit-learn pandas numpy

# Run the application
streamlit run app.py
```

Listing 1: Running the Streamlit App

The application runs locally at `http://localhost:8501`.

8 Conclusion

This project successfully developed a complete machine learning pipeline for customer churn prediction using the Telco Customer Churn dataset. The following conclusions are drawn:

Best Model: The Random Forest (Balanced) model was selected as the best model with the highest churn recall of **0.85**, correctly catching 317 out of 373 churners, and a ROC-AUC of **0.866**. Since churn prediction is a recall-driven problem where missing a churner results in direct revenue loss, recall was prioritized over raw accuracy. This model was deployed in the Streamlit application.

Class Imbalance: Using `class_weight='balanced'` was essential. Standard models without balancing achieved higher accuracy but very poor recall for the minority churn class, making them unsuitable for real business use.

Key Churn Drivers: Feature importance analysis identified *tenure*, *contract type*, *total charges*, and *internet service type* as the most significant predictors of churn.

Business Insight: Month-to-month contract customers with high monthly charges and low tenure represent the highest churn risk. Targeted retention strategies — such as offering contract upgrades or loyalty discounts to this segment — can significantly reduce churn.

Model Behavior: Learning curve analysis confirmed that all final models are well-fitted with minimal overfitting, a significant improvement over initial models that showed severe overfitting (Training F1 = 1.0, Validation F1 = 0.50).

8.1 Future Work

- Implement advanced ensemble methods such as XGBoost or LightGBM
- Experiment with SMOTE (Synthetic Minority Over-sampling Technique) for handling class imbalance
- Incorporate time-series analysis to capture churn trends over time
- Deploy the application on a cloud platform (e.g., Streamlit Cloud, Heroku)

References

1. IBM. (2019). *Telco Customer Churn Dataset*. Retrieved from Kaggle: https://www.kaggle.com/blastchar/telco-customer-churn
2. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
3. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
4. He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284.
5. Streamlit Inc. (2023). *Streamlit: The fastest way to build data apps*. https://streamlit.io
6. McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.
7. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
8. Waskom, M. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.